

Exposé for Master's Thesis

Adaptive and Fault-Tolerant Leader Election in Resource-Constrained IoT and Robotic Clusters

Pushpit Bhardwaj

Master's Thesis Proposal¹

¹Technische Universität Dresden, Chair of Distributed and Networked Systems

²Andreas-Pfitzmann-Bau, 3rd Floor , Nöthnitzer Straße 46, 01187 Dresden

September 3, 2025

1 Objective

The objective of this thesis is to design, implement, and evaluate a lightweight leader election protocol that is both adaptive and fault-tolerant, and specifically tailored for clusters of IoT devices and robotic nodes operating under severe resource constraints. Such systems are frequently subject to unreliable communication links, limited battery power, and the absence of global coordination. A key outcome of this work will be an improved leader election protocol that operates effectively under these conditions and is rigorously assessed with respect to convergence time, communication overhead, energy efficiency, and fault tolerance.

The base of this thesis is **PraSLE (Practical Self-Stabilizing Leader Election)** [4], a protocol explicitly designed for IoT devices that guarantees recovery from transient faults through self-stabilization. While PraSLE is a strong foundation, it leaves open important areas for improvement. Building on it, this thesis will introduce and evaluate several extensions: adaptive energy-aware leader election, controlled leader rotation, link-quality-aware selection, adaptive timeout tuning, and backup-based fault recovery. Together, these enhancements aim to realize a protocol that is lightweight, energy- and link-aware, and robust under practical IoT and robotic conditions.

The central research question can therefore be phrased as follows: *How can leader election be realized in a way that is both robust (fault-tolerant, self-stabilizing) and practically efficient (low-overhead, resource-aware, quickly converging) for clusters where nodes and links are unreliable, resource-limited, and potentially mobile? What, exactly, makes a “good leader” in such settings, and how should the election process reflect domain/application needs?* In this thesis, the focus is primarily on static or quasi-static networks, while full mobility support is left for future work.

Base Protocol Candidates and Choice of PraSLE

There are several candidate families of leader election protocols. Classical approaches such as Bully, Ring, or GHS are well studied but assume reliable communication and relatively powerful nodes; they are thus too heavy for IoT. Consensus protocols like Paxos or Raft offer strong fault tolerance, yet their complexity and overhead make them unsuitable for resource-constrained networks. Robotics-specific proposals (e.g., Lunar Leader or Stabilizing Flocking) are tailored to specific mission domains, while population protocols are elegant in theory but rely on random pairwise interactions that are difficult to implement in wireless IoT settings.

PraSLE is chosen as the base because it directly targets IoT devices, is lightweight in design, and offers self-stabilization. Importantly, its current limitations—no support for energy- or context-aware leader criteria—precisely match the gap this thesis seeks to address. This makes PraSLE both a solid foundation and a natural candidate for extension.

Simulator Choice

For implementation and evaluation, we selected **Contiki-NG with the Cooja simulator**. Cooja emulates constrained IoT motes and provides built-in support for energy consumption models and wireless link-quality instrumentation, enabling direct measurement of our leader-election design with minimal modeling assumptions. OMNeT++ was considered as an alternative due to its scalability and physical-layer fidelity, but it would require custom energy models and significantly greater development effort, making it less suitable for a resource-aware IoT focus.

2 Problem Analysis

Background and Motivation

Leader election (LE) is a cornerstone of coordination in distributed systems. It underpins critical functions such as routing, synchronization, data aggregation, and collective control. In IoT and robotics, leader election becomes even more crucial as systems grow in scale and autonomy.

The challenge lies in the fact that IoT devices and robotic clusters are subject to constraints and

disruptions that are far stronger than in classical distributed systems. Energy is scarce, communication is unreliable, and topologies may change due to node failures or environmental effects. In robotic systems, a leader's role can even be mission-critical: the failure of a leader may disrupt group coordination or cause system-wide inefficiency. A suitable leader, therefore, cannot be determined solely by a static property such as node ID, but should take into account metrics such as available energy, communication reliability, or suitability for the specific task.

Technical Challenges

Designing a leader election protocol for such settings requires addressing several intertwined challenges. First, the network is dynamic: nodes may join, leave, or fail unpredictably. Second, the devices are resource-constrained, with limited CPU, memory, bandwidth, and—most critically—battery capacity. Third, communication links are unreliable, with packet loss, variable latency, and frequent disconnects. Fourth, fairness and sustainability demand that leadership not be fixed but rotated in a way that avoids draining single nodes. Fifth, no global time, asynchronous scheduling, and possibly no consistent coordinate frame. Finally, robustness is crucial: the protocol must be self-stabilizing, so the system can recover automatically from transient faults and arbitrary states without external intervention ([2][3]).

Context-Aware Criteria for Leader Choice

To meet these challenges, leader election must go beyond classical criteria such as node IDs. A *context-aware* protocol may incorporate:

- **Energy-related factors:** remaining battery capacity, energy consumption rate.
- **Communication quality:** RSSI (signal strength), packet delivery ratio, connectivity degree, or latency to neighbors.
- **System load:** CPU utilization, memory availability, task-specific suitability.
- **Mobility (robotic contexts):** relative movement within the cluster or predicted position stability.
- **Reliability/security:** past availability history, trust metrics in adversarial environments.

In IoT scenarios, energy and link quality dominate; in robotics, mobility and task awareness are also critical. A leader should therefore be the node most able to *sustain* its role under the given conditions.

Nuanced Leader Election

The above considerations give rise to what we call **nuanced leader election**: a process in which the leader is not chosen on the basis of a single, static property, but by balancing multiple, dynamic, and context-dependent metrics. Unlike the Bully or Ring algorithms, which always select the node with the largest ID, a nuanced protocol adapts to the current state of the network and application. This adaptability is essential in order to achieve both robustness and efficiency in IoT and robotic clusters.

Gap

PraSLE already provides an efficient and self-stabilizing basis for IoT leader election. However, it leaves open several critical gaps: it does not yet account for energy-aware or context-aware selection criteria; it lacks mechanisms for controlled rotation to avoid depleting leaders; it does not exploit link-quality metrics essential in lossy networks; and its fault detection relies only on fixed timeouts, which are inefficient in variable environments. Addressing these limitations is the focus of this thesis.

3 Goals and Research Questions

The thesis pursues three overarching goals. First, to implement and analyze PraSLE in Contiki-NG using the Cooja simulator to establish its baseline performance under IoT/robotic conditions. Second, to extend PraSLE with targeted improvements—energy awareness, leader rotation, link-quality considerations, adaptive timeouts, and backup-based recovery. Third, to rigorously evaluate these extensions under realistic fault models and benchmark them against both the original PraSLE and classical algorithms such as Bully and Ring.

From this, several research questions arise:

1. How does PraSLE perform in practice with respect to convergence time, energy overhead, and robustness?
2. To what extent do energy- and link-quality-aware extensions improve network lifetime and leader stability?
3. How do adaptive mechanisms (timeouts, rotation, backup lists) influence responsiveness and fault recovery?
4. What trade-offs exist between communication overhead, election speed, fairness, and robustness?

4 Related Work

Leader election has been studied extensively in classical distributed systems. The Bully and Ring algorithms are widely known but assume reliable communication and relatively powerful nodes, making them unsuitable for IoT. Consensus protocols such as Paxos or Raft provide strong guarantees but at the cost of heavy message overhead.

In IoT and WSN contexts, recent work has proposed probabilistic or randomized approaches to reduce message cost, or neighbor-collaboration methods to improve robustness. Yet these typically remain weak in the face of high packet loss or are not optimized for energy efficiency. PraSLE [4] has been proposed as a practical alternative for IoT, combining lightweight design with self-stabilization. While effective, it still lacks awareness of energy and link quality.

In robotics, protocols such as Lunar Leader [1] and Stabilizing Flocking [2] incorporate mobility and

mission-specific leader roles. These are robust in their domains but less general and lightweight. Finally, population protocols [3] present a minimalistic but elegant model, showing theoretical feasibility of fast and compact leader election. However, their assumptions limit direct practical adoption.

In summary, current protocols either assume too much (reliability and resources) or ignore key practical metrics such as energy and link quality. PraSLE strikes a balance, but needs targeted extensions to meet real IoT/robotic demands.

5 Concept and Solution Approach

The concept of this thesis is to extend PraSLE in a modular fashion. Concretely:

1. Extend the state machine with energy and link metrics.
2. Modify the scoring function for leader election to weight these metrics.
3. Introduce a controlled leader handover mechanism triggered by low energy.
4. Add adaptive timeout management for more efficient failure detection.
5. Maintain a ranked backup list to ensure rapid recovery from leader loss.

These protocol extensions will be implemented in **Contiki-NG** using the **Cooja simulator**, which provides realistic support for wireless channel models, mote-level energy consumption, and protocol evaluation under interference and lossy conditions. This choice is motivated by the fact that Cooja directly emulates IoT hardware and integrates energy and link-quality measurement tools, thereby reducing the need for abstract modeling. While **OMNeT++** offers scalability and high PHY-layer fidelity, it requires additional development of custom energy models, which would divert focus from the core contributions of this thesis.

Protocol vs. Application Aspects of Leader Rotation

Leader rotation can be divided into responsibilities of the **protocol itself** and those left to the **application level**:

- **Protocol-level (inside PraSLE):**
 - State management for leader handover (handover flags, acknowledgements, atomic update of leader state).
 - Guaranteeing correctness and stability (minimum leadership term to prevent oscillations).
 - Communication primitives (e.g., a LEADER_HANDOVER message type and acknowledgement).
- **Application-level (configurable policies):**

- Ranking criteria for successors (energy, link quality, mobility, task load).
- Threshold values for triggering handover (e.g., energy < 20%).
- Size of backup list (1 for IoT motes, more for robotic teams).
- Weighting of multi-criteria scores (e.g., balancing energy vs. link quality).

In short, the protocol guarantees fault-tolerant correctness of the rotation, while applications define what constitutes a “good leader” in context.

6 Evaluation

Evaluation will be carried out in three phases: first, a baseline assessment of PraSLE in static and lossy IoT networks using Cooja; second, a comparative evaluation of PraSLE against Bully and Ring under identical conditions; and third, an in-depth analysis of each extension both individually and in combination. Metrics include convergence latency, election stability, fairness of leader rotation, energy efficiency, and network lifetime.

7 Schedule

Weeks	Task
1–3	Literature review; baseline implementation of PraSLE in Contiki-NG Cooja
4–6	Baseline experiments with PraSLE; analysis of performance under IoT/robotic scenarios in Cooja
7–9	Implementation of adaptive energy-aware election and leader rotation in Contiki-NG
10–12	Implementation of link-quality-aware election and adaptive timeout tuning
13–14	Implementation of leader backup list and gossip-based fault detection
15–17	Benchmark experiments: compare baseline vs. extended PraSLE vs. classical (Bully, Ring) in Cooja
18–19	Data analysis; trade-off discussion; parameter sweeps (energy vs. link weight)
20	Thesis write-up of methodology and results
21	Refinement, discussion, and conclusion
22	Final submission and preparation for possible publication

8 References

1. K. Albee et al., “Lunar Leader: Persistent, Optimal Leader Election for Multi-Agent Exploration Teams,” *AAMAS*, 2024.
2. D. Canepa and M. Gradinariu Potop-Butucaru, “Stabilizing flocking via leader election in robot networks,” INRIA RR-6268, 2013.
3. D. Alistarh and R. Gelashvili, “Recent Algorithmic Advances in Population Protocols,” *ACM SIGACT News*, vol. 49, no. 3, 2018.
4. A. Milani et al., “A Practical Self-Stabilizing Leader Election for Networks of Resource-Constrained IoT Devices (PraSLE),” arXiv, 2021.
5. Classic distributed algorithms textbooks covering Bully, Ring, and GHS protocols.